Critical Analysis: ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search

Kai-Yu Lu

1 Methodological Strengths

1.1 Problem framing and methodological positioning

The study targets a central weakness of outcome-only self-training for reasoning: selecting training traces solely by final-answer correctness can retain logically flawed intermediate steps, leading to low-quality supervision for improving reasoning policies and for training process-aware evaluators. The proposed approach reframes self-training as a coupled search-and-learning procedure: a tree-search policy generates higher-quality reasoning traces and step-level learning targets, and the resulting traces in turn refine both a policy model and a process-oriented value model over multiple iterations.

1.2 Search policy as a principled data curator

A key methodological contribution is treating tree search (MCTS*) not only as an inference-time verifier but also as a data generator that curates training trajectories with richer supervision than outcome filtering. This is methodologically stronger than Best-of-N filtering because it operationalizes a mechanism to explore and compare partial solutions, rather than only ranking complete solutions after generation.

1.3 Joint improvement of policy and value models across iterations

The experimental design evaluates iterative self-training for multiple LLM backbones, explicitly comparing against self-training baselines (ReSTEM and Self-Rewarding) under a consistent protocol. Table 1 shows that multi-iteration training can substantially improve average performance (Ave.) across benchmarks for several backbones, and that the relative gains depend on the model and dataset.

1.4 Evaluation breadth across math and science reasoning

The work evaluates both (i) self-training improvements on multiple benchmarks (MATH, GPQA-Diamond, CEval-Hard) and (ii) search-time verification policies on scientific reasoning datasets (SciBench and SciEval) with explicit comparisons among Self-Consistency (SC), outcome reward model (ORM) Best-of-N, process reward model (PRM) Best-of-N, and MCTS*. This broadens empirical support beyond a single benchmark family.

1.5 Transparent reporting of efficiency-related measurements

The study reports average running time for different inference-time strategies and discusses token-usage trends as search hyperparameters increase. Such reporting improves interpretability of the accuracy-cost trade-off, even when full training costs are not comprehensively detailed (see Section 2).

Table 1: Self-training results reported in Table 2 (accuracy, %). "Ave." is as reported. "0th (FS)" denotes the 0th-iteration few-shot setting.

Backbone	Setting / Method	MATH	GPQA-Diamond	CEval-Hard	Ave.
LLaMA-3-8B-Instruct					
	0th (FS)	30.00	31.31	25.66	28.99
	2nd: ReSTEM	33.52	25.25	21.71	26.83
	2nd: Self-Rewarding	33.89	26.26	23.03	27.73
	2nd: ReST-MCTS*	**34.28**	**27.78**	**25.00**	**29.02**
Mistral-7B-Instruct-v0.2 (MetaMATH)					
	0th (FS)	28.28	29.29	9.21	22.26
	2nd: ReSTEM	23.86	26.26	22.37	24.16
	2nd: Self-Rewarding	23.90	26.77	25.00	25.22
	2nd: ReST-MCTS*	**24.40**	**28.79**	**26.32**	**26.50**
SciGLM-6B					
	0th (ZS)	25.18	23.74	51.97	33.63
	2nd: ReSTEM	25.86	25.25	48.68	33.27
	2nd: Self-Rewarding	23.86	28.79	48.03	33.56
	2nd: ReST-MCTS*	**23.90**	**31.82**	**51.97**	**35.90**

2 Key Limitations

2.1 Dependence on oracle final answers limits applicability

The method relies on oracle final correct answers to infer step-level process signals, which narrows applicability to tasks with ground-truth answers available at data generation time. This assumption constrains deployment to domains where correctness can be programmatically verified or labeled, and it is not resolved by the current experimental scope.

2.2 Reporting inconsistencies and incomplete protocol specificity

Table 2 contains at least one apparent inconsistency: for LLaMA-3-8B-Instruct at the 1st iteration under ReSTEM, the reported "Ave." value (17.22) does not align with the visible per-benchmark values (30.84, 26.77, 21.05). This weakens confidence in aggregation and highlights the need for clearer definitions of averaging and careful verification of reported metrics.

Moreover, several crucial details that affect reproducibility and interpretation are not fully specified in the reported excerpts, including comprehensive training compute, full hyperparameter grids explored for self-training iterations, and sensitivity of results to search thresholds. The study indicates that code and evaluation details are released publicly, which mitigates but does not eliminate the limitation when reading the paper in isolation.

2.3 Limited error analysis on failure modes

While the quantitative results establish gains, the analysis does not provide a systematic breakdown of error categories. For reasoning tasks, it is important to distinguish whether improvements come from better search exploration, better step selection, reduced arithmetic mistakes, or reduced hallucinated intermediate claims. Without such analysis, it remains unclear which reasoning subskills are being improved and where performance remains brittle.

Table 2: Efficiency and value-model comparisons reported in Tables 6 and 7. Running time is measured under the paper's basic experiment settings.

Item	**SC (CoT+SC)**	**ORM+BoN**	**PRM+BoN**	**MCTS***
Running time (s)	41	43	73	108
MATH running time (s)	37.0	33.5	34.5	41.5

Dataset	Models	ORM	PRM	Self-Rewarding	ReST-MCTS* (Value)
SciBench	GLM4	20.2	22.0	20.2	22.9
SciBench	GPT-3.5-turbo	12.8	17.4	13.7	20.2

2.4 Computational cost scaling remains a concern

The study reports that token usage increases rapidly as the branching factor b and iteration count T rise, particularly due to prompt tokens. In addition, the search strategy is slower than simpler baselines in wall-clock time (Table 2). These costs may constrain practical use when latency or budget is limited.

2.5 Evaluation scope and generalization

The main empirical focus is on math and science reasoning datasets. It is not established whether the approach generalizes to tasks where intermediate steps are less structured, where reward signals are sparse or subjective, or where correctness cannot be verified by matching a short final answer.

3 Technical Bottlenecks

3.1 Search-policy bottlenecks: branching, depth, and value calibration

The approach hinges on accurate step-wise valuation to guide search. If the value model is miscalibrated, MCTS* may over-exploit misleading branches, producing systematically biased training data. This creates a feedback loop risk: search errors can be amplified during self-training if poor trajectories are preferentially selected and reinforced.

3.2 Coupled optimization instability

Jointly improving a policy model and a value model using self-generated data introduces distribution shift at every iteration. As the policy changes, the state distribution visited during search changes, and the value model is forced to generalize to new partial trajectories. This coupling can be unstable without explicit mechanisms for preventing drift, such as held-out validation of value targets, conservative updates, or uncertainty-aware exploration. The current results show improvements, but do not quantify stability across seeds or across a broader set of tasks.

3.3 Trade-off between verification strength and computation

Table 2 highlights a core trade-off: MCTS* is slower than SC and ORM+BoN, yet the paper argues that it can attain accuracy levels that others cannot even at higher cost. The bottleneck is therefore practical: achieving the best reported accuracy may require search settings that are infeasible for real-time use, especially as token usage grows with b and T.

Table 3: Verification-model comparison reported in Table 3 (accuracy, %) for Mistral-7B (MetaMATH). SC denotes self-consistency. ORM denotes an outcome reward model. MS denotes MATH-SHEPHERD.

Method	GSM8K	MATH500
SC	84.76	57.06
ORM	86.05	71.13
SC+ORM	84.15	72.06
MS	81.44	55.91
SC+MS	84.46	62.80
SC+ReST-MCTS* (Value)	**87.03**	**73.57**

3.4 Reliance on ground-truth answers as a supervision anchor

Because step-wise targets are anchored to oracle final answers, the method cannot directly handle open-ended problems where correctness is not crisply defined. This is not only a limitation but also a technical bottleneck: removing the oracle would require alternative verification signals (formal checkers, simulators, external tools, or consensus-based validation), none of which are integrated into the current pipeline.

4 Research Implications

4.1 Implications for LLM self-training: beyond outcome filtering

The results support the view that self-training benefits from improving the *quality* of intermediate supervision, not only increasing the quantity of correct final answers. The reported gains across multiple backbones suggest that integrating structured search and step-wise evaluation can produce more informative training traces than naive filtering.

4.2 Process supervision as a reusable interface

Comparisons among ORM, PRM, and self-critic approaches suggest that process-aware scoring can be more effective than outcome-only scoring for guiding search and selecting solutions. Table 3 shows that combining self-consistency with process-aware signals can materially improve accuracy on GSM8K and MATH500, indicating that verifiers and search policies can be composed.

4.3 Benchmark design and the cost-accuracy frontier

The appendix-style benchmarking protocol explicitly ties methods to token usage and introduces standardized comparisons among SC, BoN, DFS-style ToT, and MCTS*. This reinforces a broader implication: progress in reasoning should be reported along a cost-accuracy curve, not only at a single operating point, because different deployment contexts prioritize different trade-offs.

5 Potential Research Directions

5.1 Reducing reliance on oracle final answers

A primary direction is to replace oracle answers with verifiable signals:

- Incorporating external tools (symbolic solvers, unit checkers, code execution) to provide outcome verification without labeled answers for certain domains.

- Learning verification through distillation from stronger models or ensembles, while explicitly controlling for self-confirmation bias.

These directions directly address the strongest applicability constraint identified in the study.

5.2 Uncertainty-aware value modeling for safer self-training

Given the feedback loop risk between value errors and generated data, value models could output uncertainty estimates that modulate exploration. For example, high uncertainty could encourage broader exploration rather than premature exploitation, and low-confidence trajectories could be excluded or down-weighted when forming the self-training dataset.

5.3 Stability diagnostics and robustness evaluation

The study would benefit from protocols that quantify stability across random seeds, search hyperparameters, and dataset slices. A concrete direction is to introduce:

- Seed-sweeps with confidence intervals for iterative self-training gains.
- Ablations that vary b, T, and termination thresholds to map regimes where improvement is reliable versus unstable.

5.4 Efficiency improvements and adaptive search budgets

Since token usage grows rapidly with b and T, adaptive budget allocation could improve practicality:

- Early-exit criteria based on value convergence or disagreement reduction.
- Difficulty-aware budgets that allocate more search to hard instances and less to easy ones.

These directions align with the reported cost trends and the emphasis on cost-accuracy benchmarking.

5.5 Domain scaling via multi-domain reward/value modeling

The paper notes that a single reward model may not scale across domains and suggests training multiple reward models across reasoning domains. A concrete extension is multi-task or mixture-of-experts value modeling, which may preserve domain-specific calibration while enabling transfer of generic reasoning heuristics.

6 Conclusion

The study presents a methodologically meaningful shift for LLM self-training in reasoning: integrating process-guided tree search (MCTS*) to infer step-wise value targets and to curate higher-quality reasoning traces, thereby improving both the policy model and the value model over iterations. Empirical results indicate substantial gains for iterative self-training (Table 1) and strong verifier performance when combining self-consistency with the learned value model (Table 3). Additional experiments on scientific reasoning show that ReST-MCTS* can outperform baselines on SciEval (79.87% for GLM4 and 62.31% for GPT-3.5-turbo, as reported) and that PRM-based approaches can yield higher accuracy than ORM and self-critic value estimators (Table 2).

The most critical limitations are the dependence on oracle final answers for step-level target inference, incomplete characterization of stability and failure modes, and nontrivial computational overhead with

rapidly increasing token usage as search hyperparameters grow. The most promising research directions therefore focus on removing the oracle dependency via verifiable external signals, improving value-model calibration with uncertainty-aware mechanisms, and developing adaptive search budgeting to make the accuracy gains more accessible under realistic compute constraints.